

LECTURE NOTES FOR ER AND TRANSLATION TO RELATIONS

Business folks will ask you (the dev) to create a database for a business application (such as managing employees, keeping tracks of customers, etc.). You need to talk with them to understand the application requirements: what do they want to store in the database, what kinds of constraints, what kinds of queries they would often ask, etc.

Talking with business folks using a bunch of tables is hard. Better to talk using a language called ER.

THE BASIC OF ER LANGUAGE

In this language you ask them what entity sets and relationships among the entity sets they want to capture. An entity set is just a set of entities such as a set of students, a set of employees, a set of products, etc. A relationship between two entity sets A and B capture the associations between entities in A and entities in B. For example, if A is a set of students and B is a set of courses, then the relationship "take course" will record which students takes which courses.

An entity set must have multiple keys, which can be used to refer to entities in the set. One key will be primary key. Typically this will be the key being used most often in the future to find entities in the set.

Entity sets and relations can have attributes. Each attribute can only have an atomic value (or NULL value), for multiple reasons.

Each entity set will be translated into a single table that captures all attributes of that entity set. The primary key of the entity set becomes the primary key of the table. The relationship will be translated into a table that has the primary keys of the two entity sets (of the relationship), plus all attributes of the relationship. So if A is an entity set of students then that becomes a table. B is an entity set of courses then that becomes another table. The relationship "take course" will become a table that has the primary key of students, primary key of courses, and the attributes of this relationship (e.g., "semester").

KINDS OF RELATIONSHIPS

Binary relationships (those between two entity sets) can be many-many, many-1, 1-many, 1-1. Many-1 between A and B means an entity in A is associated with AT MOST one entity in B. This is indicated using an arrow on the ER diagram. When translating this, the table for the relationship is folded into the table for the many side (that is, for A).

If an entity in A is associated with EXACTLY one entity in B, then we indicate this using half circle on the ER diagram.

MULTIWAY RELATIONSHIPS

A relationship can be between 3 or more entity sets, such as a purchase relationship would be between person entity set, store entity set, and product entity set. The business folks will tell you if they want a relationship like this.

Consider a relationship R between three entity sets A, B, and C. When drawing an ER diagram for R, if you have an arrow going into C, that means an entity x from A and an entity y from B are associated with AT MOST one entity z from C. Unlike binary relationships, you cannot fold the table for R into the table for A, B, or C. In other words, you need a table for R, which will store the primary keys of A, B, and C, and the attributes of R, if any.

IS-A HIERARCHY AND WEAK ENTITY SETS

An is-a hierarchy can be translated using the OO, ER, or NULL approach. If B is a subclass of A, then the primary key of B is the same as the primary key of A. Intuitively, B just has some extra attributes that entities in A do not have.

Weak entity set definition and translation are straightforward as in the lectures.

CONSTRAINTS

You need to ask for as many constraints as possible from the business folks. Keys and primary keys are an important type of constraints.

Foreign keys are another important type of constraints. Foreign keys are imposed at the table level. Consider Students take Courses. Suppose you have converted Students entity set into a table A, Courses into a table B, and "take" into a table R. Suppose you know that A stores ALL students. Then clearly when table R refers to a student id, that id must occur in table A. So you can impose a foreign key constraint between A and R. For example, "the values in the column sid in Table R must occur in the column sid in Table A". Here "sid" in Table A is the primary key of Table A, and "sid" in Table R is called a foreign key of table A.

You can do a similar foreign key constraint between Table B and Table R, if Table B stores ALL courses.